

# Proyecto Aplicaciones Web 2

**COLEGIO  
UNIVERSITARIO**



## Integrantes:

- Colomba Matías Gabriel (43694674)
- Máximo Ezequiel Paz (44767857)
- Tano Busso (39080247)
- Bruno Ariza (46222763)

# TP1: Propuesta de Trabajo

## **Objetivo:**

Desarrollar una página web que permita al usuario seleccionar componentes de computadora y comprobar si hay alguna incompatibilidad entre ellos para reducir los problemas al momento de comprar y armar una computadora.

## **Descripción:**

El proyecto consiste en el desarrollo de una aplicación web que permita a los usuarios verificar la compatibilidad entre distintos componentes de una computadora.

El sistema permitirá seleccionar componentes como: CPU, Motherboard, GPU, Refrigeración de CPU, Gabinete y otros componentes de computadora.

A partir de estos datos, la aplicación determinará si son compatibles entre sí.

## **Problemas que resuelve:**

- Problemas al momento de armar la computadora.
- Incompatibilidades técnicas.
- Tiempo perdido en devoluciones o cambios.

## **Solución propuesta:**

La aplicación permitirá validar la compatibilidad entre componentes de un sistema

El sistema analiza datos como:

- Tamaño de la GPU.
- Espacio disponible en el gabinete.
- Compatibilidad de chipset de la Motherboard y la CPU

Una vez analizado los datos, devolverá resultados como: Incompatible y Advertencia (ej: “entra justo”).

## **Propuesta de Valor:**

- Reduce errores de compra.
- Ahorra tiempo y dinero
- Brinda una herramienta simple y útil.
- Permite escalar el sistema añadiendo más componentes y sus correspondientes validaciones.

## **Roles:**

- Tano Busso - Front End: diseño de interfaz, estructura HTML y estilos CSS.
- Colomba Matías - Back End: desarrollo de la API y base de datos.
- Bruno - Testing: pruebas del sistema, detección de errores en la validación de componentes.
- Máximo Paz - Administrador de Repositorio: acepta Pull Request, mantiene el repositorio acomodado

Estos roles pueden ir rotando entre los integrantes a lo largo del desarrollo del proyecto.

## **Backend del sistema:**

El proyecto no solo contempla el desarrollo de una interfaz visual, sino también la construcción de un backend que permita gestionar la lógica de negocio y los datos del sistema.

El backend será desarrollado utilizando Node.js junto con Express.js, permitiendo la creación de un servidor web capaz de:

- Servir la aplicación web al cliente
- Gestionar las solicitudes del usuario
- Procesar la lógica de compatibilidad entre componentes
- Consumir y/o exponer APIs

## Funcionalidades del backend:

El backend del sistema se encargará de:

- Validar la compatibilidad entre componentes (CPU, GPU, motherboard, etc.)
- Procesar reglas de negocio (ej: compatibilidad de chipset, espacio físico)
- Obtener datos desde APIs externas (en etapas iniciales)
- Gestionar datos propios mediante una base de datos (en etapas posteriores)

## Arquitectura prevista:

El sistema se desarrollará siguiendo una arquitectura basada en:

- Servidor backend (Node.js + Express)
- API REST para comunicación con el frontend
- Separación por capas (controladores, servicios, modelos)

Esto permitirá que el sistema sea:

- Escalable
- Mantenable
- Fácil de extender

## Evolución del proyecto:

El desarrollo se realizará en etapas, alineadas con la materia:

- TP2: Servidor backend + consumo de API externa
- TP3: Implementación de CRUD + base de datos
- TP4: Autenticación, seguridad y gestión de usuarios

## Cronograma:

Para tener registro de las fechas límite de entrega utilizamos la herramienta Trello. El siguiente es el link del cronograma:

- [https://docs.google.com/spreadsheets/d/1LWX6\\_xM2nwUQHazp-P6tRjhPdGYJuJ8B/edit?usp=sharing&oid=102938447517335443619&rtpof=true&sd=true](https://docs.google.com/spreadsheets/d/1LWX6_xM2nwUQHazp-P6tRjhPdGYJuJ8B/edit?usp=sharing&oid=102938447517335443619&rtpof=true&sd=true)

## **Trello:**

Para registrar el estado de las tareas y quien las realiza utilizamos la herramienta Trello. El siguiente es el link del Trello:

- <https://trello.com/invite/b/69ebad08e596a9275f475eb0/ATTI376df0786cd918cfaf1e3294b764f6db81FC7B75/trabajo-practico-aw2-v-20>

## **Herramienta de seguimiento y control de versiones:**

Para este proyecto usaremos Git como herramienta y Github como plataforma. El siguiente es el link del Github:

- <https://github.com/Maximo2205/Aplicaciones-web-2---Trabajos-Prcticos>

## TP2: Back-End para servir el sitio web

En esta fase del proyecto, hemos consolidado el núcleo del sistema mediante el desarrollo de un backend en **Node.js** con el framework **Express**. El objetivo principal fue establecer un servidor funcional que actúe como intermediario para la gestión de componentes y proporcione una buena base para la lógica de compatibilidad.

## Arquitectura del Sistema

Se ha implementado una arquitectura basada en la separación de responsabilidades:

- **Capa de Presentación (Frontend):** Interfaz desarrollada con HTML y CSS. La lógica de usuario se maneja mediante **ESM (ECMAScript Modules)**, facilitando la modularidad.
- **Capa de Servicio (Backend):** Servidor API RESTful que centraliza la lógica de negocio y la gestión de recursos.
- **Gestión de Estado:** Implementación de localStorage para la persistencia del lado del cliente, permitiendo que la selección del catálogo se mantenga al navegar hacia la vista de análisis.

## Motor de Compatibilidad

El software incluye un motor de reglas especializado (listComp.mjs) que analiza requisitos técnicos críticos:

- **Lógica de Compatibilidad:** Cruce de datos entre sockets y chipsets de Motherboard y CPU.
- **Estándares de Memoria y Almacenamiento:** Verificación de generaciones DDR y formatos M.2.
- **Restricciones Físicas:** Comprobación de dimensiones espaciales, como la longitud de la GPU frente a la capacidad del Gabinete (Chassis).

## Estado de Tareas (Metodología Kanban)

- **Finalizado:** Estructura de navegación, diseño UI/UX del catálogo, motor de compatibilidad inicial, servidor base y API CRUD completa.
- **En Proceso:** Integración final del frontend con el servidor local (transición de MockAPI al servidor propio).
- **Pendiente:** Implementación de Base de Datos relacional (TP3) y Sistema de Autenticación y Seguridad (TP4).

## Conclusión y Cumplimiento de Consigna

El servidor se encuentra operativo y cumple con el requerimiento académico de servir el proyecto frontend. La estructura actual permite que la interfaz consulte y envíe datos de forma eficiente, sentando las bases para la futura migración a una base de datos persistente.

# TP3:

## Creación del CRUD para administrar Front-End

## Resumen de la Etapa y Objetivos

En esta tercera fase del proyecto, el sistema **chkCompatibility** experimentó una evolución estructural crítica, transitando desde un esquema con datos aislados en memoria hacia una arquitectura robusta de producción. Los pilares alcanzados en esta entrega comprenden:

- **Migración de Arquitectura:** Reestructuración del backend adoptando el patrón de diseño **MVC (Modelo-Vista-Controlador)**.
- **Contenerización y Entorno:** Implementación de **Docker**.
- **Persistencia Avanzada:** Conexión del servidor Express a un motor de **Base de Datos** relacional de manera local (PostgreSQL).

## Reestructuración de la Arquitectura (Patrón MVC)

El proyecto fue reestructurado al Patrón MVC visto en clases, siendo una carpeta para los módulos y dentro de esta mismas carpetas individuales de cada módulo, de momento estando la carpeta “Componentes” con los archivos .mjs del modelo, controlador y rutas.

- **Modelo (modelo.componentes.mjs):** Capa encargada de la comunicación directa con la base de datos PostgreSQL utilizando la abstracción de consultas SQL.
- **Controlador (controlador.componentes.mjs):** Intermediario lógico que recibe los objetos de petición (req) y respuesta (res) de Express. Se encarga de procesar los parámetros, y responder al cliente con los códigos de estado HTTP estandarizados correspondientes.
- **Rutas (rutas.componentes.mjs):** Enrutador específico encargado de mapear los verbos HTTP correspondientes a los endpoints de la API REST y delegar su ejecución al controlador.

## Implementación de MockAPI y Estrategia de Integración

Temporalmente utilizamos una API diseñada en MockAPI que funcionara como una base de datos provisional en donde pudiéramos integrar los datos mostrados en la página web. La API usada para el proyecto se encuentra en el siguiente link:

'<https://69f3f9acbd2396bf5310826e.mockapi.io/api/v1/componente>'

Actualmente descartamos la API y la reemplazamos por una base de datos diseñada e integrada en el programa por medio del sistema Docker.

## Relación entre MockAPI y la Base de Datos Integrada en Docker

El uso de MockAPI no reemplaza a la base de datos persistente, sino que actúa como su **esquema espejo de preproducción**. La coexistencia e integración de ambas tecnologías se define bajo los siguientes lineamientos técnicos:

- **Homologación de Modelos:** El JSON estructurado en MockAPI (que incluye campos críticos como name, tipo, cpuSocket, ramDdr, entre otros) fue el plano de diseño utilizado para estructurar las tablas y entidades dentro de la base de datos relacional montada bajo Docker.
- **Flujo de Migración de Datos:** El backend desarrollado bajo arquitectura MVC ya cuenta con los endpoints (GET)
- **Estrategia de Desacoplamiento (Swapping):** Al haber estructurado las funciones del frontend utilizando variables dinámicas (fetch(url)), el paso definitivo de producción consiste únicamente en modificar la constante url en los archivos .mjs para que apunten al entorno local dockerizado (<http://localhost:3000/api/v1/componentes>), garantizando que la

aplicación siga funcionando idénticamente pero con persistencia real y local.

## Entorno de Desarrollo y Contenerización con Docker

Para asegurar la portabilidad y homogeneidad del sistema entre todos los integrantes del equipo, la infraestructura local se orquestó mediante contenedores independientes:

1. **Contenedor del Servidor (Express):** Ejecuta la lógica MVC en Node.js, sirviendo tanto los archivos estáticos del frontend en el puerto 3000 como respondiendo a las llamadas de la API interna.
2. **Contenedor de Base de Datos:** Instancia aislada del motor relacional que gestiona la persistencia real del catálogo de hardware.
3. **Pool de Conexiones (conexion.db.mjs):** La comunicación entre el servidor y el contenedor de la base de datos se implementó mediante la clase Pool de la librería pg.

## Documentación Técnica de los Endpoints (API REST)

A partir de la implementación de las rutas modularizadas, la API REST expone los siguientes endpoints funcionales vinculados a operaciones de la base de datos:

| Método | Endpoint                | Descripción Técnica del Flujo de Datos                                |
|--------|-------------------------|-----------------------------------------------------------------------|
| GET    | /api/v1/componentes     | Recupera la lista completa de componentes desde la tabla componentes. |
| GET    | /api/v1/componentes/:id | Recupera un componente en específico desde la tabla componentes.      |

|               |                         |                                                                                                                                                    |
|---------------|-------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>POST</b>   | /api/v1/componentes     | Inserta un nuevo registro de hardware en la base de datos a través del controlador, enviando los datos parametrizados para evitar inyecciones SQL. |
| <b>PUT</b>    | /api/v1/componentes/:id | Modifica un componente existente según su id. Si el elemento no existe en las filas afectadas de la base de datos, se controla la respuesta.       |
| <b>DELETE</b> | /api/v1/componentes/:id | Elimina de forma física un registro basándose en su clave primaria (id).                                                                           |

## Estado de Tareas (Metodología Kanban)

- **Finalizado:** Estructura de navegación, diseño UI/UX del catálogo, motor de compatibilidad inicial, servidor base, implementación de la API, testing y validación, desarrollo del CRUD de componentes, diseño de la base de datos, migración de datos a la base de datos.
- **En Proceso:** Implementación de Login y Seguridad.
- **Pendiente:** Seguridad y protección de rutas y Testing y validación de la Seguridad.

## Conclusión

Actualmente el proyecto se encuentra en una fase avanzada, contando con su propia base de datos (en principio fue una API) hecha en Docker, y con sus propias variables de entorno. También se implementó las funciones CRUD en una nueva pestaña específicamente dirigida a los administradores de la página, que les permitirá la manipulación de los datos de la misma. Próximamente nos enfocaremos en la implementación de un Login y las distintas variables de seguridad.

# TP4:

## Implementación de autenticación, accesos y seguridad

## Planificación de Seguridad y Autenticación

Para la siguiente fase de desarrollo, el sistema incorporará mecanismos de control de acceso y protección de datos para restringir las operaciones CRUD únicamente a usuarios administradores autenticados. Basándonos en la arquitectura planificada, se implementará un sistema de autenticación basado en **JSON Web Tokens (JWT)** y cookies seguras.

## Flujo de Autenticación Proyectado

El proceso de resguardo de rutas y validación de sesiones seguirá el siguiente esquema técnico:

1. **Autenticación del Usuario (Login):** Se dispondrá de un endpoint de tipo POST encargado de recibir las credenciales del usuario. El servidor validará la existencia del usuario en la base de datos y comprobará la coincidencia de las contraseñas (las cuales serán procesadas mediante hashing).
2. **Generación y Firma del Token (JWT):** Tras una validación exitosa, el servidor utilizará la librería `jsonwebtoken` para firmar un token asíncronamente mediante el método `jwt.sign()`. El *payload* del token encapsulará datos útiles no sensibles, como el identificador del usuario y su rol dentro del sistema. La seguridad de la firma estará respaldada por una clave secreta declarada de forma aislada en el entorno (`process.env.FIRMA_JWT`).
3. **Persistencia mediante Cookies Seguras:** En lugar de almacenar el token en el almacenamiento local del navegador (lo que expone al sistema a ataques XSS), el servidor inyectará el JWT directamente en una cookie firmada utilizando atributos de seguridad avanzados:
  - `http://localhost:3000/login` → Endpoint que inyecta la cookie tras el login.

- httpOnly: true → Impide que scripts maliciosos de terceros accedan al token desde el frontend.
- secure: true → Asegura que la cookie viaje únicamente bajo conexiones cifradas HTTPS.
- sameSite: 'lax' → Protege la sesión contra ataques de falsificación de peticiones en sitios cruzados (CSRF).

## Middleware de Control de Acceso (comprobarToken)

Se desarrollará una función middleware dedicada que interceptará las peticiones a rutas protegidas. Su comportamiento lógico será el siguiente:

- El middleware extraerá la cookie firmada enviada de forma automática por el navegador.
- Mediante el método `jwt.verify()`, comprobará la validez y vigencia del token.
- **Flujo de Autorización:** Si el token es válido y el payload confirma que el usuario posee los permisos requeridos (rol de administrador), se invocará a la función `next()` para dar luz verde a la ruta estática de administración (`/admin`) o a los endpoints de modificación de hardware (POST, PUT, DELETE). En caso de fallo o token expirado, el servidor cortará el flujo de ejecución inmediatamente y redirigirá al usuario a la interfaz de inicio de sesión (`/login`).

## Estado de Tareas (Metodología Kanban en Trello)

Para llevar el control y visibilidad del proyecto, seguimos empleando nuestro tablero de **Trello** estructurado bajo el modelo **Kanban**. Con el cierre de la infraestructura de datos y la planificación de la seguridad, el estado actual de nuestras tarjetas se distribuye de la siguiente manera:

- **Pendiente:** Planificación de las pruebas de penetración y tests de estrés sobre los endpoints protegidos una vez que el sistema de seguridad esté implementado por completo.
- **En Proceso:** Desarrollo del módulo de autenticación en el backend. Esto incluye la creación de las interfaces de vista para el /login y el panel de /admin, la programación del controlador de sesiones y la definición de las variables de entorno en el archivo .env (FIRMA\_JWT).
- **Listo / Hecho (Done):** Refactorización a arquitectura **MVC Modular** por carpetas, contenerización completa de la aplicación y de la base de datos **PostgreSQL** mediante **Docker**, persistencia e inserción del set de datos semilla con arreglos nativos (TEXT[]).

## Conclusión

El proyecto **chkCompatibility** ha alcanzado su madurez estructural. La migración hacia un modelo **MVC** modularizado, la portabilidad absoluta provista por **Docker** y el salto a una base de datos real con **PostgreSQL** sientan los cimientos ideales para la fase actual. La integración de la estrategia de autenticación por **JWT** y el resguardo de rutas mediante cookies y middlewares de control permitirán concluir la próxima entrega (TP4) cumpliendo con los estándares de seguridad, escalabilidad y robustez técnica requeridos en ambientes de producción reales.